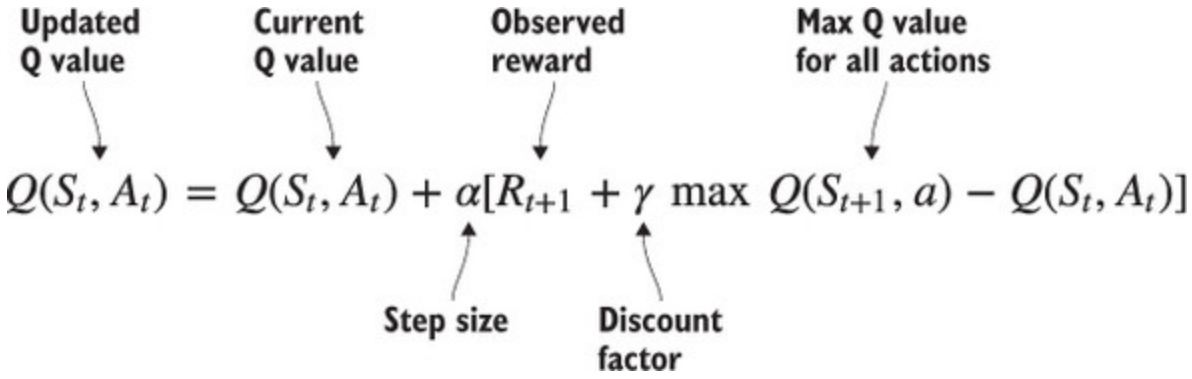
the actual reward, and updated the network accordingly. That was a particular and simple implementation of a broader class of Q-learning algorithms that is described by the following update rule:

Table 3.1. Q-learning update rule

Math



Pseudocode

```
def get_updated_q_value(old_q_value, reward, state, step_size, discount):
     term2 = (reward + discount * max([Q(state, action) for action in
              actions])
     term2 = term2 – old_q_value
     term2 = step_size * term2
     return (old_q_value + term2)
```

English

The Q value at time *t* is updated to be the current predicted Q value plus the amount of value we expect in the future, given that we play optimally from our current state.

3.2.2. Tackling Gridworld

You've now seen the formula for Q-learning. Let's take a step back and apply this formula to our Gridworld problem. Our goal in this chapter is to train a neural network to play a simple Gridworld game from scratch. All the agent will have access to is what the board looks like, just as a human player would; the algorithm has no informational advantage. Moreover, we're starting with an untrained algorithm, so it literally knows nothing at all about the world. It has no prior information about how games work. The only thing we'll provide is the reward for reaching the goal. The fact that we will be able to teach the algorithm to learn to play, starting from nothing, is actually quite impressive.